Lab 7: Motor Control of the Pololu 3pi+ Robot using Pulse-Width Modulation

10/28/2026

Daniel Xu
ECE 3218L

1 Overview

The Pololu 3pi+ 32U4 is a small two-wheeled robot built around an ATmega32U4 running at 16 MHz. Each wheel is driven by a brushed DC motor through its own TI DRV8838 motor driver. Each driver has two control inputs from the microcontroller: a direction pin (PH) that selects forward or reverse, and an enable pin (EN) that switches the motor on. When EN is low, the driver brakes the motor by shorting its terminals.

This lab controlled the motors in two ways. In the first, the enable pins were used as plain digital outputs, so each motor either ran at full speed or braked. In the second, Timer1 generated phase-correct pulse-width modulation (PWM) on the enable pins, switching each motor between driving and braking fast enough that it ran at a fraction of full speed. The program waits for button A, runs a set of full-speed tests on each motor and then both motors, and finally runs the same moves at quarter speed with PWM. In lab, the course-provided OLED driver was used to show the current step on the robot's display, and the quarter-speed tests were repeated with each of the five Timer1 prescalers to find the best PWM frequency.

2 Procedure

The programs were written in C and built with avr-gcc and a Makefile instead of Microchip Studio. They were loaded over the robot's USB port through its bootloader, using avrdude. The program is split into three modules: `motors.c` drives the motor pins and Timer1, `button.c` reads button A, and `main.c` runs the test sequence. The full code is in Appendix A.

1. **Setup.** Table 1 lists the Port B pins used by the motor drivers and button A, taken from the Pololu 3pi+ 32U4 User's Guide. All four motor pins were made outputs and set low, which brakes both motors. Button A connects PB3 to ground when pressed, so PB3 was made an input with its internal pull-up enabled: it reads high when released and low when pressed.

Table 1: Port B pins used in this lab.

Pin	Function	Notes
PB1	Right motor direction (PH)	
PB2	Left motor direction (PH)	
PB3	Button A	Input, pull-up, low when pressed
PB5	Right motor enable (EN)	OC1A
PB6	Left motor enable (EN)	OC1B

Table 2 shows how the DRV8838 responds to its inputs. Setting EN high runs the motor, and PH picks the direction. Combining the two tables gives the Port B bit patterns for each move in Table 3. Only the four motor bits (mask 0x66) were ever changed, so that writing a motor pattern does not turn off the pull-up on PB3 or disturb PB0, which the OLED uses as its data/command pin.

Table 2: DRV8838 behavior with nSLEEP high.

EN	PH	Result
0	X	Brake (both outputs low)
1	0	Forward
1	1	Reverse

Table 3: Port B motor bits for each move (bits 6, 5, 2, 1 are L EN, R EN, L PH, R PH).

Move	Binary	Hex
Brake	00000000	0x00
Left forward	01000000	0x40
Left reverse	01000100	0x44
Right forward	00100000	0x20
Right reverse	00100010	0x22
Both forward	01100000	0x60
Both reverse	01100110	0x66
Spin CCW (left reverse, right forward)	01100100	0x64
Spin CW (left forward, right reverse)	01100010	0x62

The program does nothing until button A is pressed and released (Code 1), which gives time to unplug the USB cable and set the robot down. Each wait is checked again after 20 ms so that switch bounce is not counted as a second press. The robot starts moving 500 ms after the button is released.

Code 1: Waiting for a debounced press and release of button A (button.c).

```
void button_a_wait(void)
{
    /* Wait for a press that stays pressed through the bounce time */
    do {
        while (!button_a_pressed())
            ;                   /* loop indefinitely until pressed */
        _delay_ms(20);          /* debounce                        */
    } while (!button_a_pressed());

    /* Wait for release, again debounced */
    do {
        while (button_a_pressed())
            ;
        _delay_ms(20);
    } while (button_a_pressed());
}
```

2. **Full-speed tests.** For the full-speed tests, TCCR1A was cleared so that Timer1 does not drive PB5 and PB6, and the enable pins were set and cleared directly through PORTB. The function `motors_set_full()` takes a state (brake, forward, or reverse) for each motor. Each move ran for 1 s, followed by a brake and a 3 s pause. The program first ran each motor alone in each direction (left forward, left reverse, right forward, right reverse) and then both together (forward, reverse, spin CCW, spin CW). Every move went in the expected direction, so the direction pins did not need to be flipped from the default of PH low for forward. [confirm, and note anything unexpected]

3. **Quarter-speed tests with PWM.** Timer1 was then set to Mode 1, 8-bit phase-correct PWM, with TOP = 0xFF (Code 2). In non-inverting mode, TCCR1A = 0xA1: COM1A1 and COM1B1 connect OC1A (right motor) and OC1B (left motor) to the timer, and WGM10 selects Mode 1 together with WGM13:12 = 00 in TCCR1B. The low three bits of TCCR1B (CS12:10) select the prescaler and start the timer.

Code 2: Switching to phase-correct PWM (motors.c).

```
TCCR1A = (1 << COM1A1) | (1 << COM1B1) | (1 << WGM10);
if (pwm_inverting)
    TCCR1A |= (1 << COM1A0) | (1 << COM1B0);

/* TCCR1B: WGM13:12 = 00, CS12:10 = prescaler (starts the timer) */
TCCR1B = (uint8_t)prescale & 0x07;
```

In phase-correct mode the counter counts up from 0 to TOP and back down, so one PWM period is $2 \cdot \mathrm{TOP}$ timer clocks, and the PWM frequency is

$$f_{\mathrm{PWM}} = \frac{f_{\mathrm{clk}}}{2 \cdot N \cdot \mathrm{TOP}} = \frac{16\ \mathrm{MHz}}{2 \cdot N \cdot 255}, \tag{1}$$

where N is the prescaler. With non-inverting output, the duty cycle is OCR1x/255, so quarter speed is OCR1x = 64 (25.1%). The function `motors_set_speed()` takes a signed speed from −255 to 255 for each motor: the sign sets the direction pin and the magnitude is written to OCR1A or OCR1B. Each quarter-speed move (forward, reverse, spin CCW, spin CW) ran for 2 s, followed by a brake and a 3 s pause.

4. **OLED display.** The course-provided `font.c`, `OLED.c` and `OLED.h` were first tested with the program from the handout, built as its own project. Two corrections were needed. The handout calls `OLED_Init()`, but the provided driver names the function `OLED_init()`. With the string changed to `"ECE3218"`, the display showed only ECE321: the provided `OLED_writeText()` always draws exactly 6 characters, so the seventh was cut off. It also never checks for the string's terminating `'\0'`, so a shorter string would show whatever bytes follow it in memory.

   `OLED_writeText()` was modified to draw characters until the end of the string and then fill the rest of the line with spaces (Code 3). Text starts at column 16 and each character is 7 pixels wide (5 for the glyph and 2 for the gap), so 16 characters fit across the 128-pixel display. The blank fill also erases a longer previous message without clearing the whole display, which is slow because the driver bit-bangs every byte. After this change, ECE3218 displayed in full.

Code 3: Modified character loop of OLED_writeText() (OLED.c).

```
// Draw the string up to its '\0', then fill the rest of the row with
// spaces so any leftovers from a longer previous message are erased.
for (uint8_t i=0;i<OLED_TEXT_MAX;i++)
{
    char glyph=' ';
    if (*text) glyph= *text++;
```

In the motor program, the OLED shows the step currently running (for example "L fwd", "1/4CCW" or "Brake"), which made it easy to match each motion of the robot to the code that caused it.

5. **Prescaler experiment.** After the first quarter-speed run, the program shows "A=next" and waits for button A. Each press moves to the next prescaler (wrapping from 1024 back to 1), shows its value on the OLED (for example "N=8"), and repeats the quarter-speed tests. Only the CS12:10 bits of TCCR1B are rewritten, so the rest of the PWM setup is unchanged. Table 4 lists the PWM frequency from Equation 1 and the observed behavior for each prescaler.

Table 4: Quarter-speed behavior for each Timer1 prescaler.

N	CS12:10	f_{PWM}	Period	Observed behavior
1	001	31.4 kHz	31.9 μs	Quiet, no audible whine. Moved well.
8	010	3.92 kHz	255 μs	Smoothest motion, smoother than $N = 1$, but with a high-pitched whine that the other settings did not have.
64	011	490 Hz	2.04 ms	[observations]
256	100	123 Hz	8.16 ms	Somewhat rattly.
1024	101	30.6 Hz	32.6 ms	Very rattly.

3 Discussion

3.1 Unexpected Behaviors

The main surprise was that $N = 8$ felt smoother than $N = 1$ at quarter speed, even though $N = 1$ was expected to be smoothest. This is explained in Section 3.3. The OLED truncating ECE3218 to ECE321 was also unexpected at first, until reading `OLED_writeText()` showed that it always draws 6 characters. [any other unexpected behaviors from lab, e.g. drifting on straight moves]

3.2 Non-Inverting and Inverting PWM

In phase-correct mode, the compare output is updated each time TCNT1 passes OCR1x. In non-inverting mode (COM1x1:0 = 10), OC1x is cleared on a compare match while counting up and set on a match while counting down. The output is therefore high while TCNT1 < OCR1x, and the duty cycle is OCR1x/255. In inverting mode (COM1x1:0 = 11), the actions are swapped: the output is high while TCNT1 > OCR1x, and the duty cycle is 1 − OCR1x/255.

Since EN high drives the motor and EN low brakes it, the two modes map OCR values to speed in opposite directions. In non-inverting mode, OCR1x = 0 brakes, 64 is quarter speed, and 255 is full speed. In inverting mode, OCR1x = 255 brakes, 191 is quarter speed, and 0 is full speed. With the same OCR value of 64, inverting mode would run the motors at about three-quarter speed, and an OCR of 0, meant as a brake, would run them at full speed. For that reason, the program converts every duty cycle through one function that subtracts it from 255 in inverting mode, so the rest of the code works the same way in both modes.

Once the OCR values are adjusted, the motor itself cannot tell the two modes apart: both produce a symmetric pulse at the same frequency and duty cycle. The only difference is where the pulse sits in the timer cycle, centered on BOTTOM in non-inverting mode and on TOP in inverting mode. Non-inverting mode is the more natural choice here because a larger OCR means a faster motor, and the reset value of 0 means stopped. [note what was seen if PWM_INVERTING = 1 was tried on the robot]

3.3 Choosing the Prescaler

The behaviors in Table 4 follow from how the PWM period compares to how fast the motor current can change, and from where the PWM frequency falls in the range of human hearing.

At $N = 1$, the 31.4 kHz PWM frequency is above the roughly 20 kHz limit of human hearing, so it made no audible sound. The 31.9 μs period is also short compared to how fast the winding inductance lets the motor current change, so the current, and therefore the torque, stays nearly constant through each cycle.

At $N = 8$, the 3.92 kHz PWM frequency falls in the 2–5 kHz range where the ear is most sensitive. The windings and the motor's body vibrate at the PWM frequency, so the motor acted like a small speaker and produced the whine. The motion felt smoother than at $N = 1$, most likely because of static friction. At $N = 1$, the current is almost constant at about a quarter of its full-speed value, which may be barely enough to keep the gearbox turning. At $N = 8$, the current ripples more during each cycle, and its peaks are well above the average. Those torque peaks help break static friction, so the motion is more consistent at low speed. This advantage should shrink at higher duty cycles, where the average torque is already well above friction.

At $N = 256$ and $N = 1024$, the periods of 8.16 ms and 32.6 ms are long enough for the current to rise and fall a large amount within each cycle. The motor gets a torque pulse during each on-time and is braked during each off-time, which is the rattle that was heard and felt. At 30.6 Hz the robot is effectively kicked 30 times a second, so $N = 1024$ was the worst. [$N = 64$: expected a lower-pitched hum at 490 Hz; compare with what was observed]

$N = 1$ was chosen as the best prescaler. It is the only setting that is both silent and free of torque ripple, and the whine at $N = 8$ is a real drawback for a robot that runs for long periods. The smoothness advantage of $N = 8$ comes from low duty cycles, where the torque is close to the friction limit, and should disappear at the higher speeds the robot uses most. Pololu's own motor library for the 3pi+ makes the same choice: it runs Timer1 with no prescaler and a TOP of 400, giving 20 kHz PWM, which is just above the range of hearing. [optional: distance traveled in the 2 s quarter-speed forward run at each N, to support the friction explanation]

4 Conclusion

This lab controlled the 3pi+ robot's two motors through Port B, first as on/off outputs for full-speed tests and then with Timer1 phase-correct PWM on OC1A and OC1B for quarter-speed tests. The OLED driver was modified to display strings of up to 16 characters, and it showed each step

of the test sequence. Of the five prescalers, $N = 1$ (31.4 kHz) was the best because it is silent and gives the steadiest motor current, while $N = 256$ and $N = 1024$ made the robot rattle and $N = 8$ produced an audible whine.

A Full Code

Code 4: Test sequence and prescaler experiment (main.c).

```
/*
 * main.c - ECE 2218 Lab 7: Motor Control of the Pololu 3pi+ 32U4 using PWM
 *
 * Sequence (matches the "Before the Lab" steps):
 *   1. Setup: brake motors, wait for button A.
 *   2. Full-speed test, one motor at a time.
 *   3. Full-speed test, both motors together.
 *   4. Quarter-speed tests using Timer1 phase-correct PWM.
 * After that, each press of button A switches to the next Timer1
 * prescaler and repeats step 4, for the in-lab prescaler experiment.
 */
#ifndef F_CPU
#define F_CPU 16000000UL
#endif

#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>
#include "motors.h"
#include "button.h"

/* ---- Settings ------------------------------------------------------- */

/* 1 = show each step on the OLED (needs font.c, OLED.c, OLED.h in this
 * folder). Set to 0 to build without them.                            */
#define USE_OLED 1

/* Prescaler for the first run of part 4 (see pwm_prescale_t in motors.h) */
#define START_PRESCALE  PRESCALE_64

/* 0 = non-inverting PWM, 1 = inverting PWM (for the post-lab question) */
#define PWM_INVERTING   0

#define RUN_FULL_MS   1000   /* full-speed moves: 1 s                  */
#define RUN_PWM_MS    2000   /* quarter-speed moves: 2 s               */
#define PAUSE_MS      3000   /* "brake and wait a few seconds"         */

/* ---- OLED helper ---------------------------------------------------- */
#if USE_OLED
#include "OLED.h"

static char text[50];

/* Show one short message on the top line of the OLED.
 * OLED_writeText() draws up to OLED_TEXT_MAX (16) characters and blanks
 * the rest of the line, so leftovers from the previous message disappear
 * without clearing the whole screen. Longer messages are cut off.       */
static void show(const char *msg)
{
```

```
    snprintf(text, sizeof text, "%s", msg);
    OLED_writeText(text);
}

static void oled_setup(void)
{
    OLED_init();                /* note: lowercase i in the provided OLED.c */
    OLED_reset();
    OLED_clearDisplayRam();
    OLED_configureDefault();
    show("Lab 7");
}
#else
static void show(const char *msg) { (void)msg; }   /* does nothing */
static void oled_setup(void) { }
#endif

/* ---- Timing helpers ------------------------------------------------- */

/* _delay_ms() wants a constant, so loop 1 ms at a time */
static void delay_ms(uint16_t ms)
{
    while (ms--)
        _delay_ms(1);
}

/* Brake both motors and wait */
static void brake_and_wait(void)
{
    motors_brake();
    show("Brake");
    delay_ms(PAUSE_MS);
}

/* ---- Part 2: full speed, one motor at a time ----------------------- */
static void part2_single_motors(void)
{
    show("L fwd");
    motors_set_full(MOTOR_FORWARD, MOTOR_BRAKE);
    delay_ms(RUN_FULL_MS);
    brake_and_wait();

    show("L rev");
    motors_set_full(MOTOR_REVERSE, MOTOR_BRAKE);
    delay_ms(RUN_FULL_MS);
    brake_and_wait();

    show("R fwd");
    motors_set_full(MOTOR_BRAKE, MOTOR_FORWARD);
    delay_ms(RUN_FULL_MS);
    brake_and_wait();

    show("R rev");
    motors_set_full(MOTOR_BRAKE, MOTOR_REVERSE);
    delay_ms(RUN_FULL_MS);
    brake_and_wait();
}

/* ---- Part 3: full speed, both motors ------------------------------- */
```

```c
static void part3_both_motors(void)
{
    show("Fwd");
    motors_set_full(MOTOR_FORWARD, MOTOR_FORWARD);
    delay_ms(RUN_FULL_MS);
    brake_and_wait();

    show("Rev");
    motors_set_full(MOTOR_REVERSE, MOTOR_REVERSE);
    delay_ms(RUN_FULL_MS);
    brake_and_wait();

    /* Spinning CCW (seen from above): left wheel back, right wheel forward */
    show("CCW");
    motors_set_full(MOTOR_REVERSE, MOTOR_FORWARD);
    delay_ms(RUN_FULL_MS);
    brake_and_wait();

    /* Spinning CW: left wheel forward, right wheel back */
    show("CW");
    motors_set_full(MOTOR_FORWARD, MOTOR_REVERSE);
    delay_ms(RUN_FULL_MS);
    brake_and_wait();
}

/* ---- Part 4: quarter speed with PWM ------------------------------------ */
static void part4_quarter_speed(void)
{
    const int16_t q = MOTOR_SPEED_QUARTER;

    show("1/4fwd");
    motors_set_speed(q, q);
    delay_ms(RUN_PWM_MS);
    brake_and_wait();

    show("1/4rev");
    motors_set_speed(-q, -q);
    delay_ms(RUN_PWM_MS);
    brake_and_wait();

    show("1/4CCW");
    motors_set_speed(-q, q);
    delay_ms(RUN_PWM_MS);
    brake_and_wait();

    show("1/4 CW");
    motors_set_speed(q, -q);
    delay_ms(RUN_PWM_MS);
    brake_and_wait();
}

/* Label for the OLED, e.g. "N=64" */
static const char *prescale_label(pwm_prescale_t p)
{
    switch (p) {
        case PRESCALE_1:    return "N=1";
        case PRESCALE_8:    return "N=8";
        case PRESCALE_64:   return "N=64";
        case PRESCALE_256:  return "N=256";
```

```
        case PRESCALE_1024: return "N=1024";
        default:            return "N=?";
    }
}

int main(void)
{
    /* ---- Part 1: setup ---- */
    motors_init();          /* motor pins as outputs, both braked */
    button_a_init();        /* PB3 input with pull-up             */
    oled_setup();

    show("Push A");
    button_a_wait();        /* don't move until button A is pressed */
    delay_ms(500);          /* time to get your hand clear          */

    part2_single_motors();
    part3_both_motors();

    /* Switch from digital mode to Timer1 PWM */
    pwm_prescale_t prescale = START_PRESCALE;
    motors_pwm_init(prescale, PWM_INVERTING);
    show(prescale_label(prescale));
    delay_ms(1000);
    part4_quarter_speed();

    /* ---- Prescaler experiment: A = next prescaler, repeat part 4 ---- */
    for (;;) {
        show("A=next");
        button_a_wait();

        prescale = (prescale >= PRESCALE_1024) ? PRESCALE_1 : prescale + 1;
        motors_pwm_set_prescale(prescale);

        show(prescale_label(prescale));
        delay_ms(1000);
        part4_quarter_speed();
    }
}
```

Code 5: Motor driver interface (motors.h).

```
/*
 * motors.h - Motor control for the Pololu 3pi+ 32U4 (ECE 2218 Lab 7)
 *
 * Motor driver pins (Pololu 3pi+ 32U4 User's Guide, section 5.9):
 *   PB1  Right motor direction (DIR)
 *   PB2  Left  motor direction (DIR)
 *   PB5  Right motor speed     (OC1A, Timer1 PWM)
 *   PB6  Left  motor speed     (OC1B, Timer1 PWM)
 *
 * Each motor has a DRV8838 driver: the speed pin enables the motor,
 * the direction pin picks forward or reverse. Speed pin LOW = brake.
 *
 * Two modes:
 *   Digital mode (Lab parts 2-3): speed pins are plain on/off outputs,
 *                                 so the motors run at full speed or brake.
 *   PWM mode     (Lab part 4):    Timer1 drives the speed pins with
```

```
 *                                phase-correct PWM for variable speed.
 */
#ifndef MOTORS_H
#define MOTORS_H

#include <stdint.h>

/* Direction / state for digital (full-speed) mode */
typedef enum {
    MOTOR_BRAKE = 0,
    MOTOR_FORWARD,
    MOTOR_REVERSE
} motor_state_t;

/* Timer1 clock prescaler choices (CS12:CS10 bits in TCCR1B).
 * PWM freq (8-bit phase correct) = 16 MHz / (2 * N * 255)          */
typedef enum {
    PRESCALE_1    = 1,   /* ~31.4 kHz */
    PRESCALE_8    = 2,   /* ~3.92 kHz */
    PRESCALE_64   = 3,   /* ~490 Hz   */
    PRESCALE_256  = 4,   /* ~123 Hz   */
    PRESCALE_1024 = 5    /* ~30.6 Hz  */
} pwm_prescale_t;

/* Full speed is 255 in PWM mode (8-bit, TOP = 255) */
#define MOTOR_SPEED_MAX     255
#define MOTOR_SPEED_QUARTER  64

/* Set up all four motor pins as outputs and brake both motors.
 * Leaves the motors in digital (full-speed on/off) mode.          */
void motors_init(void);

/* Digital mode: run each motor at full speed forward/reverse, or brake. */
void motors_set_full(motor_state_t left, motor_state_t right);

/* Switch to PWM mode: Timer1 phase-correct 8-bit PWM on OC1A/OC1B.
 * inverting = 0 for non-inverting output, 1 for inverting output.  */
void motors_pwm_init(pwm_prescale_t prescale, uint8_t inverting);

/* Change the PWM prescaler without touching anything else. */
void motors_pwm_set_prescale(pwm_prescale_t prescale);

/* PWM mode: speed from -255 (full reverse) to +255 (full forward).
 * 0 brakes that motor.                                             */
void motors_set_speed(int16_t left, int16_t right);

/* Brake both motors (works in either mode). */
void motors_brake(void);

#endif /* MOTORS_H */
```

Code 6: Motor driver: full-speed and PWM modes (motors.c).

```
/*
 * motors.c - Motor control for the Pololu 3pi+ 32U4 (ECE 2218 Lab 7)
 * See motors.h for the pin map and an overview.
 */
#include <avr/io.h>
```

```c
#include "motors.h"

/* ---- Pin definitions (all on Port B) ---------------------------------- */
#define R_DIR  PB1   /* right motor direction            */
#define L_DIR  PB2   /* left  motor direction            */
#define R_PWM  PB5   /* right motor speed, OC1A          */
#define L_PWM  PB6   /* left  motor speed, OC1B          */

/* Direction pin level for FORWARD. Pololu's library drives DIR low for
 * forward, so 0 here. If a motor turns the wrong way on your robot,
 * change that motor's value to 1.                                      */
#define L_FWD_LEVEL 0
#define R_FWD_LEVEL 0

static uint8_t pwm_inverting = 0;   /* remembered from motors_pwm_init() */

/* Set a direction pin for forward (reverse = 0) or reverse (reverse = 1). */
static void set_dir(uint8_t pin, uint8_t fwd_level, uint8_t reverse)
{
    uint8_t level = fwd_level ^ reverse;   /* flip the level for reverse */
    if (level)
        PORTB |=  (1 << pin);
    else
        PORTB &= ~(1 << pin);
}

/* Value to load into OCR1x for a duty of 0..255.
 * Non-inverting: output is HIGH for the first OCR counts -> OCR = duty.
 * Inverting:     output is LOW  for the first OCR counts -> OCR = 255 - duty.
 * Either way, duty 0 holds the pin LOW (brake) and 255 holds it HIGH.     */
static uint8_t duty_to_ocr(uint8_t duty)
{
    return pwm_inverting ? (uint8_t)(255 - duty) : duty;
}

void motors_init(void)
{
    /* Disconnect Timer1 from the pins so PORTB controls them directly */
    TCCR1A = 0;
    TCCR1B = 0;

    /* All four motor pins low (= brake), then make them outputs */
    PORTB &= ~((1 << R_DIR) | (1 << L_DIR) | (1 << R_PWM) | (1 << L_PWM));
    DDRB  |=  ((1 << R_DIR) | (1 << L_DIR) | (1 << R_PWM) | (1 << L_PWM));
}

/* One motor in digital mode: speed pin HIGH = full speed, LOW = brake. */
static void set_full_one(uint8_t dir_pin, uint8_t pwm_pin,
                         uint8_t fwd_level, motor_state_t state)
{
    if (state == MOTOR_BRAKE) {
        PORTB &= ~(1 << pwm_pin);
    } else {
        set_dir(dir_pin, fwd_level, state == MOTOR_REVERSE);
        PORTB |= (1 << pwm_pin);
    }
}

void motors_set_full(motor_state_t left, motor_state_t right)
```

```
{
    set_full_one(L_DIR, L_PWM, L_FWD_LEVEL, left);
    set_full_one(R_DIR, R_PWM, R_FWD_LEVEL, right);
}

void motors_pwm_init(pwm_prescale_t prescale, uint8_t inverting)
{
    pwm_inverting = inverting ? 1 : 0;

    /* Start with both motors braked */
    OCR1A = duty_to_ocr(0);
    OCR1B = duty_to_ocr(0);

    /* TCCR1A:
     *   COM1A1:0 and COM1B1:0 = 10 non-inverting, 11 inverting
     *   WGM11:10 = 01  -> with WGM13:12 = 00 this is Mode 1,
     *                     "PWM, Phase Correct, 8-bit", TOP = 0x00FF   */
    TCCR1A = (1 << COM1A1) | (1 << COM1B1) | (1 << WGM10);
    if (pwm_inverting)
        TCCR1A |= (1 << COM1A0) | (1 << COM1B0);

    /* TCCR1B: WGM13:12 = 00, CS12:10 = prescaler (starts the timer) */
    TCCR1B = (uint8_t)prescale & 0x07;
}

void motors_pwm_set_prescale(pwm_prescale_t prescale)
{
    TCCR1B = (TCCR1B & ~0x07) | ((uint8_t)prescale & 0x07);
}

/* Clamp a signed speed to -255..255 */
static int16_t clamp_speed(int16_t s)
{
    if (s >  MOTOR_SPEED_MAX) return  MOTOR_SPEED_MAX;
    if (s < -MOTOR_SPEED_MAX) return -MOTOR_SPEED_MAX;
    return s;
}

void motors_set_speed(int16_t left, int16_t right)
{
    left  = clamp_speed(left);
    right = clamp_speed(right);

    set_dir(L_DIR, L_FWD_LEVEL, left  < 0);
    set_dir(R_DIR, R_FWD_LEVEL, right < 0);

    OCR1B = duty_to_ocr((uint8_t)(left  < 0 ? -left  : left));   /* left  = OC1B */
    OCR1A = duty_to_ocr((uint8_t)(right < 0 ? -right : right));  /* right = OC1A */
}

void motors_brake(void)
{
    if (TCCR1A & ((1 << COM1A1) | (1 << COM1B1))) {
        /* PWM mode: 0% duty holds both speed pins low */
        OCR1A = duty_to_ocr(0);
        OCR1B = duty_to_ocr(0);
    } else {
        /* Digital mode */
        PORTB &= ~((1 << R_PWM) | (1 << L_PWM));
```

```
    }
}
```

Code 7: Button A interface (button.h).

```
/*
 * button.h - User pushbutton A on the Pololu 3pi+ 32U4 (ECE 2218 Lab 7)
 *
 * Button A is on PB3. It connects the pin to ground when pressed, so we
 * turn on the ATmega32U4's internal pull-up: released = HIGH, pressed = LOW.
 */
#ifndef BUTTON_H
#define BUTTON_H

#include <stdint.h>

/* Make PB3 an input with its pull-up enabled. */
void button_a_init(void);

/* 1 if button A is held down right now, 0 if not. */
uint8_t button_a_pressed(void);

/* Loop until button A is pressed and released (with debouncing).
 * The robot starts moving only after you let go of the button.   */
void button_a_wait(void);

#endif /* BUTTON_H */
```

Code 8: Button A input with debouncing (button.c).

```
/*
 * button.c - User pushbutton A on the Pololu 3pi+ 32U4 (ECE 2218 Lab 7)
 */
#ifndef F_CPU
#define F_CPU 16000000UL
#endif

#include <avr/io.h>
#include <util/delay.h>
#include "button.h"

#define BTN_A  PB3

void button_a_init(void)
{
    DDRB  &= ~(1 << BTN_A);   /* input                          */
    PORTB |=  (1 << BTN_A);   /* enable internal pull-up        */
    _delay_us(10);            /* let the pin settle high        */
}

uint8_t button_a_pressed(void)
{
    return (PINB & (1 << BTN_A)) == 0;   /* LOW = pressed */
}

void button_a_wait(void)
{
    /* Wait for a press that stays pressed through the bounce time */
```

```
    do {
        while (!button_a_pressed())
            ;                   /* loop indefinitely until pressed */
        _delay_ms(20);          /* debounce                        */
    } while (!button_a_pressed());

    /* Wait for release, again debounced */
    do {
        while (button_a_pressed())
            ;
        _delay_ms(20);
    } while (button_a_pressed());
}
```

Code 9: Course-provided OLED driver header, with OLED_TEXT_MAX added (OLED.h).

```
/*
 * OLED.h
 *
 * Created: 9/19/2024 7:12:26 PM
 *  Author: sternbal
 */


#ifndef OLED_H_
#define OLED_H_

#define OLEDCLK PD3 // D1
#define OLEDRES PD2 // D0
#define OLEDDC PB0 // D17
#define OLEDMOSI PD5
// Characters per line of OLED_writeText(): text starts at column 16 and
// each character is 7 pixels wide (5 glyph + 2 gap), so 16 fit in 128.
#define OLED_TEXT_MAX 16
void OLED_clk_low(void);
void OLED_clk_high(void);
void OLED_mos_pin(uint8_t);
void OLED_write(uint8_t);
void OLED_commandMode(void);
void OLED_dataMode(void);
void OLED_reset(void);
void OLED_clearDisplayRam(void);
void OLED_configureDefault(void);
void OLED_writeText(char*);
void OLED_init(void);
uint8_t getGlyphColumn(uint8_t, uint8_t);
#define SET_COL_ADDR_LOW 0x00
#define SET_COL_ADDR_HIGH 0x10
#define SET_CONTRAST 0x81
#define SET_SEG_REMAP 0xA0
#define SET_INV_DISP 0xA6
#define SET_DISPLAY_ON 0xAE
#define SET_PAGE_ADDR 0xB0
#define SET_COM_SCAN_DIR 0xC0

#endif /* OLED_H_ */
```

Code 10: Course-provided OLED driver, with OLED_writeText() modified (OLED.c).

```c
/*
 * OLED.c
 *
 * Created: 9/19/2024 9:50:58 PM
 *  Author: sternbal
 */
#include <avr/io.h>
#include "OLED.h"
#define F_CPU 16000000L
#include <util/delay.h>
#include <avr/pgmspace.h>

extern const PROGMEM uint8_t pololuOledFont[][5];


void OLED_init(void)
{
    // set reset pin to output
    DDRD |= 1<<OLEDRES;

    // set clock pin to output and make it low
    DDRD |= 1<<OLEDCLK;
    PORTD &= ~(1<<OLEDCLK);

    // set MOSI pin to output
    DDRD |= 1<<OLEDMOSI;


    // set data/command pin to output
    DDRB |= 1<<OLEDDC;

}

void OLED_clk_low(void)
{
    PORTD &= ~(1<<OLEDCLK);

}
void OLED_clk_high(void)
{
    PORTD |= (1<<OLEDCLK);

}

void OLED_mos_pin(uint8_t val)
{
    if(val==0) PORTD &= ~(1<<OLEDMOSI);
    else PORTD |= 1<<OLEDMOSI;
}

void OLED_reset(void)
{
    PORTD &= ~(1<<OLEDRES);
    _delay_us(10);
    PORTD |= (1<<OLEDRES);
    _delay_us(10);
}
```

```
58
59
60  void OLED_clearDisplayRam(void)
61  {
62      OLED_clk_low();
63      OLED_commandMode();
64      OLED_write(SET_COL_ADDR_LOW | 2);
65      for (uint8_t page = 0;page < 8; page++)
66      {
67          OLED_commandMode();
68          OLED_write(SET_PAGE_ADDR | page);
69          OLED_write(SET_COL_ADDR_HIGH | 0);
70          OLED_dataMode();
71          for (uint8_t i = 0 ; i < 128; i++)
72          {
73              OLED_write(0);
74          }
75      }
76  }
77
78  void OLED_configureDefault(void)
79  {
80      OLED_clk_low();
81      OLED_commandMode();
82      OLED_write(SET_SEG_REMAP | 1);
83      OLED_write(SET_COM_SCAN_DIR | 8);
84      OLED_write(SET_CONTRAST);
85      OLED_write(0xFF);
86      OLED_write(SET_DISPLAY_ON | 1);
87  }
88
89  void OLED_commandMode(void)
90  {
91      PORTB &= ~(1<<OLEDDC);
92  }
93
94  void OLED_dataMode(void)
95  {
96      PORTB |= (1<<OLEDDC);
97  }
98
99
100 void OLED_write(uint8_t d)
101 {
102     OLED_clk_low();
103     OLED_mos_pin(d >> 7 & 0x01);
104     OLED_clk_high();
105
106     OLED_clk_low();
107     OLED_mos_pin(d >> 6 & 0x01);
108     OLED_clk_high();
109
110     OLED_clk_low();
111     OLED_mos_pin(d >> 5 & 0x01);
112     OLED_clk_high();
113
114     OLED_clk_low();
115     OLED_mos_pin(d >> 4 & 0x01);
116     OLED_clk_high();
```

```
117
118     OLED_clk_low();
119     OLED_mos_pin(d >> 3 & 0x01);
120     OLED_clk_high();
121
122     OLED_clk_low();
123     OLED_mos_pin(d >> 2 & 0x01);
124     OLED_clk_high();
125
126     OLED_clk_low();
127     OLED_mos_pin(d >> 1 & 0x01);
128     OLED_clk_high();
129
130     OLED_clk_low();
131     OLED_mos_pin(d >> 0 & 0x01);
132     OLED_clk_high();
133 }
134
135 void OLED_writeText(char *text)
136 {
137     OLED_commandMode();
138     OLED_write(SET_PAGE_ADDR | 0);
139     OLED_write(SET_COL_ADDR_HIGH | 1);
140     OLED_write(SET_COL_ADDR_LOW | 0);
141     OLED_dataMode();
142     // Draw the string up to its '\0', then fill the rest of the row with
143     // spaces so any leftovers from a longer previous message are erased.
144     for (uint8_t i=0;i<OLED_TEXT_MAX;i++)
145     {
146         char glyph=' ';
147         if (*text) glyph= *text++;
148
149         for (uint8_t pixelX=0;pixelX<5;pixelX++)
150         {
151             uint8_t column=getGlyphColumn(glyph,pixelX);
152             OLED_write(column);
153 //          OLED_write(column);
154         }
155         OLED_write(0);
156         OLED_write(0);
157     }
158 }
159 uint8_t getGlyphColumn(uint8_t glyph, uint8_t pixelX)
160 {
161     if(glyph>=0x20)
162     {
163         return pgm_read_byte(&pololuOledFont[glyph - 0x20][pixelX]);
164     }
165     return(0);
166 }
```

Code 11: OLED test from the handout (oled_test/main.c).

```
1  /*
2   * Step 1: Test out the OLED display
3   */
4  #ifndef F_CPU
5  #define F_CPU 16000000UL
```

```
#endif

#include <avr/io.h>
#include <stdio.h>      /* for snprintf() */
#include "OLED.h"

int main(void)
{
    char text[50];              // define an array to hold text to be written to the display

    OLED_init();                // set up the pins for input/output that connect to the display
                                // (the handout says OLED_Init, but the provided OLED.c names
    it OLED_init)
    OLED_reset();               // toggle the display's reset pin
    OLED_clearDisplayRam();     // display data is held in RAM. Zero everything to blank
    display
    OLED_configureDefault();    // set mode, contrast, etc
    snprintf(text, 8, "%s", "ECE3218");   // copy a string constant into our variable
    OLED_writeText(text);       // display the string contents

    while (1)
        ;                       // stop here so the text stays on screen
}
```